Smart Campus Route Finder

Pragati Engineering College, Surampalem

Subject: Advanced Data Structure and Algorithm Analysis

Department: CSE - D Year 02 Sem 01

Project: Shortest route finder using Dijkstra's Algorithm

Student Names:
25A31A05ER - R.L.Sathvika
25A31A05EC - G.Thraitheshwari
25A31A05FI - G.PrudhviRaj
25A31A05FJ - G.Vivek

Introduction & Problem Statement

Problem

Several pathways connect campus buildings. Which walking route between two places is the shortest?

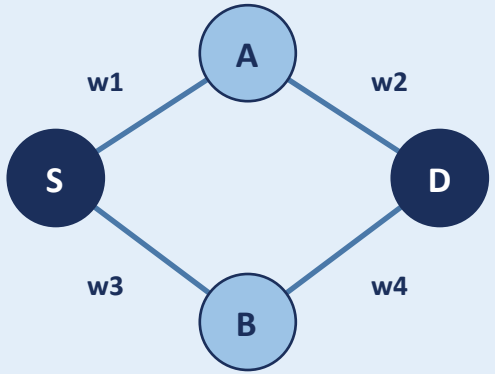
Why a graph?

Buildings become vertices and pathways become weighted edges, so shortest-path algorithms apply directly.

Objective

Find the shortest walking route between two campus locations using Dijkstra's Shortest Path Algorithm.

Alternative routes between two locations



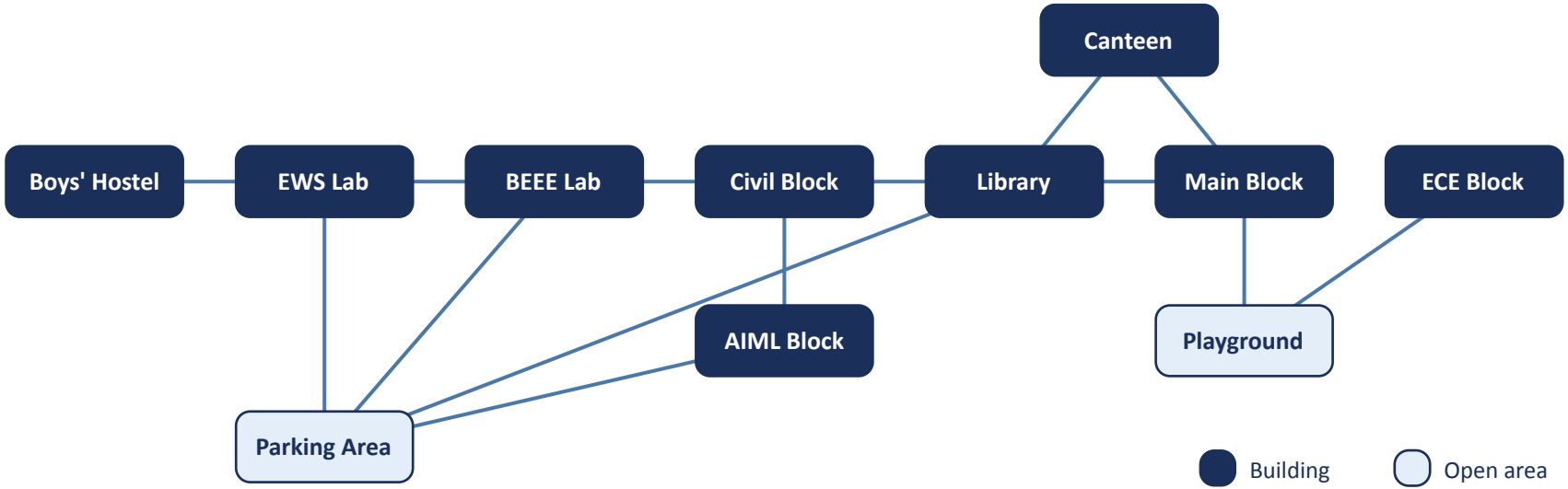
S = source, D = destination, w = distance in metres. Dijkstra picks the route with the smallest total.

Proposed System & Campus Representation

Vertex: a campus building or location (11)

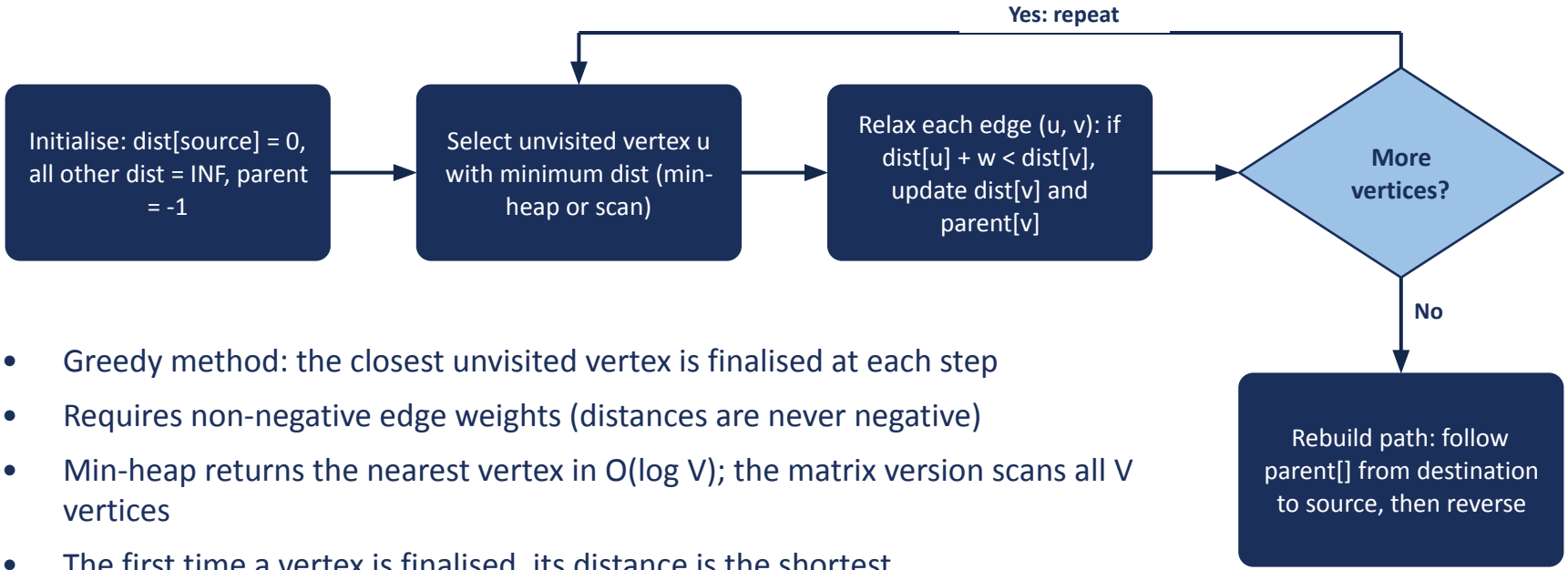
Edge: a walkable pathway (14)

Weight: distance in metres



Simplified layout from the campus photograph, not to scale. Edge weights are distances in metres, to be filled with measured values.

Dijkstra's Algorithm & Working



- Greedy method: the closest unvisited vertex is finalised at each step
- Requires non-negative edge weights (distances are never negative)
- Min-heap returns the nearest vertex in O(log V); the matrix version scans all V vertices
- The first time a vertex is finalised, its distance is the shortest

Data Structures & Implementation

Adjacency List

vector of (neighbour, distance) pairs for each vertex. Space O(V + E).

Adjacency Matrix

2D array of distances; INF marks no direct edge. Space $O(V^2)$.

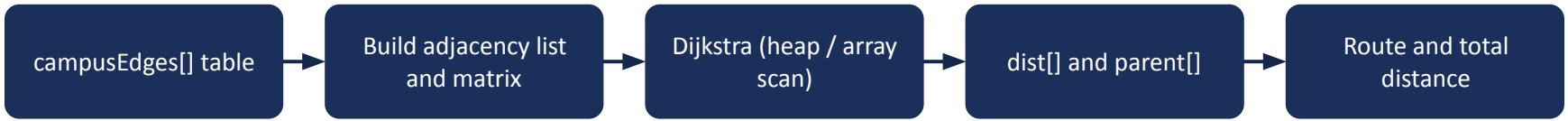
Priority Queue

STL min-heap of (distance, vertex) pairs; gives the nearest vertex in O(log V).

dist[] and parent[]

dist[] = best distance so far; parent[] = previous vertex, used to rebuild the route.

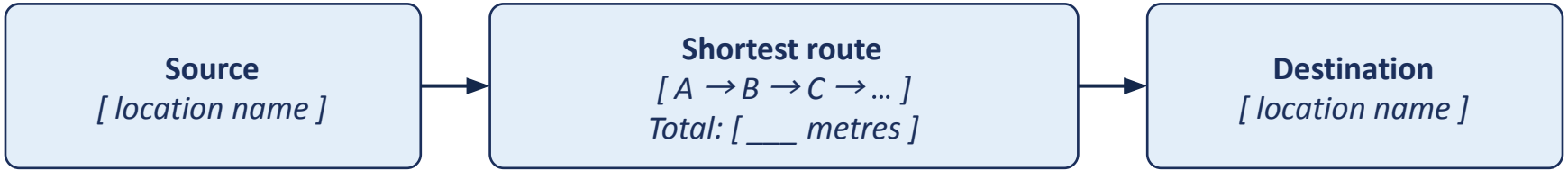
How the components work together



Language: C++ (STL: vector, priority_queue, chrono for timing)

Results & Performance Analysis

Example query (menu option 3): fill in from the program output



Parameter	Adjacency list + min-heap	Adjacency matrix + linear scan
Time complexity	O((V+E) log V)	O(V²)
Space (graph storage)	O(V + E)	O(V²)
Shortest distance (m)	*[____ m]*	*[____ m]*
Execution time (µs)	*[____ µs]*	*[____ µs]*

Record the bracketed values from menu option 5 (same graph and source, many repeated runs). Both versions must report the same shortest distance; for a graph this small, both timings are very short.

Conclusion

- Console application finds the shortest walking route between any two of 11 campus locations
- Dijkstra's greedy algorithm implemented with an adjacency list (min-heap) and an adjacency matrix
- Graph algorithms are used in navigation, network routing and mapping software
- Learned: graph modelling, edge relaxation, path reconstruction with parent[], complexity comparison
- Next step: replace sample distances with measured campus distances

Thank You